

南京信息工程大学

《数字图像处理实践》课程设计

题 目 图像增强与复原算法综合应用

学生姓名 张正珂

学 号 202483290476

院 系 计算机学院

专 业 计算机科学与技术

任课教师 姜胜芹

二〇二六年

1 图像增强任务

1.1 任务描述

图像增强处理：设计空间域与频率域结合的图像增强算法，处理图 1 中带噪声图像，去除噪声，提高图像质量。

(1) 已知：噪声为随机噪声和周期噪声混合噪声；

(2) 要求：

a) 处理带噪声图像，图 1.1 左边清晰图像仅作为算法评估使用；

b) 去噪处理后，使用均方误差、信噪比、结构相似度等图像质量度量指标评估所设计算法的性能；

图像如图 1.1 所示，左边为不含噪声的原始图像，右边为待处理的带噪声图像。



图 1.1 待处理图像

1.2 问题分析

从待恢复图看噪声为周期噪声和高斯噪声，其在傅里叶频谱中形成成对出现的孤立亮峰伴随频谱中较宽范围的高频能量抬升。程序使用 `radial_median` 对频谱进行径向中值统计，并通过 `smoothdata` 得到较平滑的 `noisyRad`。对于高频部分，代码用 `blindDrop` 参数压低盲估计参考包络，使超过参考包络的能量被判定为噪声成分。

周期噪声的估计通过 `Log_noisy` 减去大尺度高斯平滑背景 `SpecBg` 得到 `PeakScore`，再使用 `imregionalmax` 寻找局部极大值。为了避免误伤图像主体结构，程序屏蔽低频中心区域 $D < 25$ ，并要求峰值明显高于径向背景。

1.3 算法设计

算法总体流程为：读入图像并灰度归一化，计算中心化 FFT，估计径向频谱背景，构造径向压制滤波器 H_{rad} ，检测周期噪声峰并构造陷波滤波器 H_{peak} ，将二者相乘得到总频域滤波器 H_{total} ，使用原相位和修正后的幅度谱重建图像，再进行 Wiener、双边或引导滤波以及对比度调整。

径向压制部分以 $NoisyEnv$ 与 $TargetEnv$ 的差值为依据，计算 $excess = \max(0, NoisyEnv - TargetEnv - margin)$ ，再由 $H_{rad} = \exp(-refStrength * excess)$ 得到增益，并用 $minGain$ 约束最低增益，防止高频细节被完全压掉。低频中心区域 $D < 18$ 被强制保留，以维持主体亮度和结构。

周期峰陷波部分按 $PeakScore$ 从高到低选择 K 个异常峰。对每个峰及其共轭对称位置构造高斯形陷波器，峰中心增益由 $peakGain$ 控制，陷波宽度由 $D0$ 控制。这样可以针对周期条纹或周期噪声的频域特征进行定点抑制。



图 1.2 图像增强算法框图

1.4 算法实现

本实验的实现按照“读图与频谱分析、径向频谱压制、周期峰陷波、频域重建、空域后处理与指标评价”的顺序展开。下面按算法流程穿插核心代码说明。

步骤 1：读入带噪图像，完成灰度化、归一化和中心化 FFT。该步骤得到幅度谱、相位谱和对数幅度谱，为后续频域滤波提供基础。

```
I_noisy = im2double(imread('dogDistorted.bmp'));  
if ndims(I_noisy) == 3, I_noisy = rgb2gray(I_noisy); end
```

```
F_noisy = fftshift(fft2(I_noisy));
Mag_noisy = abs(F_noisy);
Phase_noisy = angle(F_noisy);
Log_noisy = log(1 + Mag_noisy);
```

步骤 2: 估计径向频谱背景并构造 H_rad。程序先对对数频谱平滑, 再按频谱中心半径统计中值, 得到 noisyRad; 随后构造盲参考包络 blindRad, 并对超过参考包络的部分进行指数衰减。

```
Log_noisy_s = imgaussfilt(Log_noisy, 0.8);
noisyRad = radial_median(Log_noisy_s, R, maxR);
noisyRad = smoothdata(noisyRad, 'gaussian', 21);

blindRad = noisyRad - blindDrop * rho.^1.15;
targetRad = blindRad;

NoisyEnv = noisyRad(R + 1);
TargetEnv = targetRad(R + 1);
excess = max(0, NoisyEnv - TargetEnv - margin);

H_rad = exp(-refStrength * excess);
H_rad = max(H_rad, minGain);
H_rad(D < 18) = 1;
```

这段代码的关键是只从带噪图像自身估计参考包络, refMix=0 时不会把清晰图像作为处理输入。H_rad 的作用是压低异常高频能量, 同时保留低频中心区域, 避免主体亮度和大结构被破坏。

步骤 3: 检测周期噪声峰并构造 H_peak。周期噪声在频谱上表现为局部亮峰, 因此程序使用大尺度高斯平滑结果作为背景, 计算 PeakScore 后选取局部极大值。

```
SpecBg = imgaussfilt(Log_noisy, 12);
PeakScore = Log_noisy - SpecBg;
PeakScore = mat2gray(PeakScore);
PeakScore(D < 25) = 0;
PeakScore(Log_noisy < NoisyEnv + 0.08) = 0;

localMax = imregionalmax(PeakScore);
idx = find(localMax);
[~, order] = sort(PeakScore(idx), 'descend');
idx = idx(order(1:K));
```

找到异常峰后, 程序对每个峰及其共轭对称位置构造高斯陷波器。这样既能抑制周期噪声, 又能保持频谱的对称性, 避免重建图像出现额外失真。

```
D1 = (X - c).^2 + (Y - r).^2;
```

```

notch1 = 1 - (1 - peakGain) * exp(-D1 / (2 * D0^2));

r2 = 2 * crow - r;
c2 = 2 * ccol - c;
D2 = (X - c2).^2 + (Y - r2).^2;
notch2 = 1 - (1 - peakGain) * exp(-D2 / (2 * D0^2));

H_peak = H_peak .* notch1 .* notch2;
H_peak(D < 18) = 1;

```

步骤 4: 合成总滤波器并进行频域重建。总滤波器由径向压制和周期陷波相乘得到，重建时保留原图相位，只修正幅度谱。

```

H_total = H_rad .* H_peak;
H_total(D < 18) = 1;

F_new = Mag_noisy .* H_total .* exp(1i * Phase_noisy);
I_freq = real(ifft2(ifftshift(F_new)));
I_freq = clip01(I_freq);

```

步骤 5: 进行轻量空域后处理和灰度动态范围调整。Wiener、双边或引导滤波用于降低残余噪声，imadjust 用于改善整体对比度。

```

I_wiener = wiener2(I_freq, [3 3]);
I_bilat = imbilatfilt(I_wiener, 0.026, 2.6);
I_final = 0.58 * I_bilat + 0.42 * I_freq;

lowHigh = stretchlim(I_final, [0.015 0.995]);
I_contrast = imadjust(I_final, lowHigh, [0 1], 1.08);
I_final = 0.35 * I_contrast + 0.65 * I_final;
I_final = clip01(I_final);

```

步骤 6: 利用清晰图像计算评价指标，并输出最终图像。dogOriginal.bmp 只在评价阶段参与 MSE、PSNR、SSIM 计算，不参与主去噪流程。

```

mseNoisy = immse(I_noisy, I_clean);
psnrNoisy = psnr(I_noisy, I_clean);
ssimNoisy = ssim(I_noisy, I_clean);

mseFinal = immse(I_final, I_clean);
psnrFinal = psnr(I_final, I_clean);
ssimFinal = ssim(I_final, I_clean);

imwrite(I_freq, 'dog_nearblind_freq.png');
imwrite(I_final, 'dog_nearblind_final.png');
saveas(compareFig, 'dog_metrics_compare.png');

```

1.5 程序运行结果及分析



图 1.3 图像增强最终结果与分析

从算法机理看，径向频谱压制能够削弱随机噪声造成的整体高频抬升，周期峰陷波能够针对周期干扰进行定点衰减。空域轻后处理进一步降低残余噪声并改善局部平滑性。算法不依赖清晰图作为处理输入，符合近盲去噪要求；不足是 K 、 $D0$ 、 $peakGain$ 等参数会影响细节保留和噪声抑制之间的平衡，需要结合图像频谱和评价指标微调。

2 图像复原任务

2.1 任务描述

设计算法复原图像图 2.1 中的降质图像，降质原因不明，请编程实现算法并使用图像质量度量指标评估算法性能。



图 2.1 待处理图像

2.2 问题分析

观察图像，降质主要被为运动模糊。运动模糊可表示为 $G(u, v) = H(u, v)F(u, v) + N(u, v)$ ，其中 G 为退化图像频谱， F 为原图频谱， H 为点扩散函数 PSF 的频率响应， N 为噪声项。

由于没有提供清晰参考图，算法采用人工估计的运动模糊参数：LEN=3，THETA=45，并设置 NSR=0.08 作为噪声信号功率比。该设置说明程序认为模糊长度较短、方向约为 45 度，同时需要一定正则化来抑制直接逆滤波可能带来的噪声放大。

2.3 算法设计

算法采用 Wiener 滤波。先根据估计的运动长度和角度构造 $PSF = fspecial('motion', LEN, THETA)$ ，再用 $deconvwnr(I_blur, PSF, NSR)$ 对每个通道做频域复原。Wiener 滤波在逆滤波基础上引入噪声约束，可以在恢复边缘的同时避免过度放大高频噪声。

复原后，程序先将像素裁剪到 $[0, 1]$ ，再使用 `edgetaper` 缓解去卷积造成的边界振铃，最后用 `imadjust` 和 `stretchlim` 做对比度增强，使复原结果的亮度层次更明显。

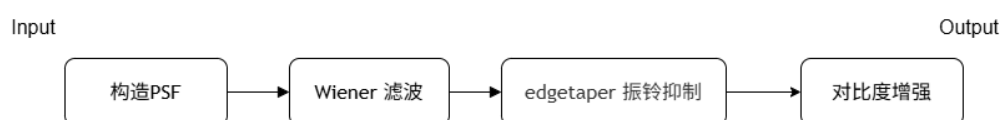


图 2.2 图像复原算法框图

2.4 算法实现

本实验的实现按照“读取降质图像、判断彩色通道、构造运动模糊 PSF、执行 Wiener 去卷积、后处理并保存结果”的顺序展开。核心代码集中在主程序和 `restore_channel` 函数中。

步骤 1：读取 `blurred wood.bmp` 并转为 `double` 类型。若图像为 RGB 彩色图，则拆分为三个通道分别处理。

```
I_blur = imread("blurred wood.bmp");  
I_blur = im2double(I_blur);
```

```

if size(I_blur, 3) == 3
    I_blur_R = I_blur(:,:,1);
    I_blur_G = I_blur(:,:,2);
    I_blur_B = I_blur(:,:,3);

```

步骤 2: 对每个颜色通道调用 `restore_channel` 进行复原, 再把三个复原后的通道重新合成为彩色图像。这样可以避免直接灰度化造成颜色信息丢失。

```

    I_restored_R = restore_channel(I_blur_R);
    I_restored_G = restore_channel(I_blur_G);
    I_restored_B = restore_channel(I_blur_B);

    I_restored = cat(3, I_restored_R, I_restored_G, I_restored_B);
else
    I_restored = restore_channel(I_blur);
end

```

步骤 3: 在 `restore_channel` 中估计运动模糊点扩散函数。代码使用 `fspecial('motion', LEN, THETA)` 构造 PSF, 其中 `LEN` 控制模糊长度, `THETA` 控制模糊方向。

```

function I_restored = restore_channel(I_blur)
    [M, N] = size(I_blur);

    LEN = 3;
    THETA = 45;
    PSF = fspecial('motion', LEN, THETA);

```

步骤 4: 使用 Wiener 滤波完成去卷积复原。NSR 表示噪声信号功率比, 用于在锐化复原和噪声抑制之间取得平衡。

```

NSR = 0.08;
I_restored = deconvwnr(I_blur, PSF, NSR);

```

步骤 5: 对复原结果进行后处理。先把像素值约束在 $[0, 1]$, 再使用 `edgetaper` 减弱边缘振铃, 最后通过 `imadjust` 增强对比度。

```

I_restored = max(0, min(1, I_restored));
I_restored = edgetaper(I_restored, PSF);
I_restored = imadjust(I_restored, stretchlim(I_restored), []);
end

```

步骤 6: 显示和保存复原结果。最终图像保存为 `forest_restored.png`, 后续报告中可将其与 `blurred wood.bmp` 并排展示。

```

figure('Name', '图像复原结果');
subplot(1,2,1); imshow(I_blur); title('模糊图像');
subplot(1,2,2); imshow(I_restored); title('维纳滤波复原');

```



```
imwrite(I_restored, 'forest_restored.png');
```

2.5 程序运行结果及分析



图 2.3 图像复原前后对比

算法优点是实现简洁、计算量低、能直接处理彩色图像；不足是 PSF 参数依赖人工经验，如果 LEN、THETA 或 NSR 与实际退化不匹配，可能出现复原不足、细节过锐或振铃增强等问题。